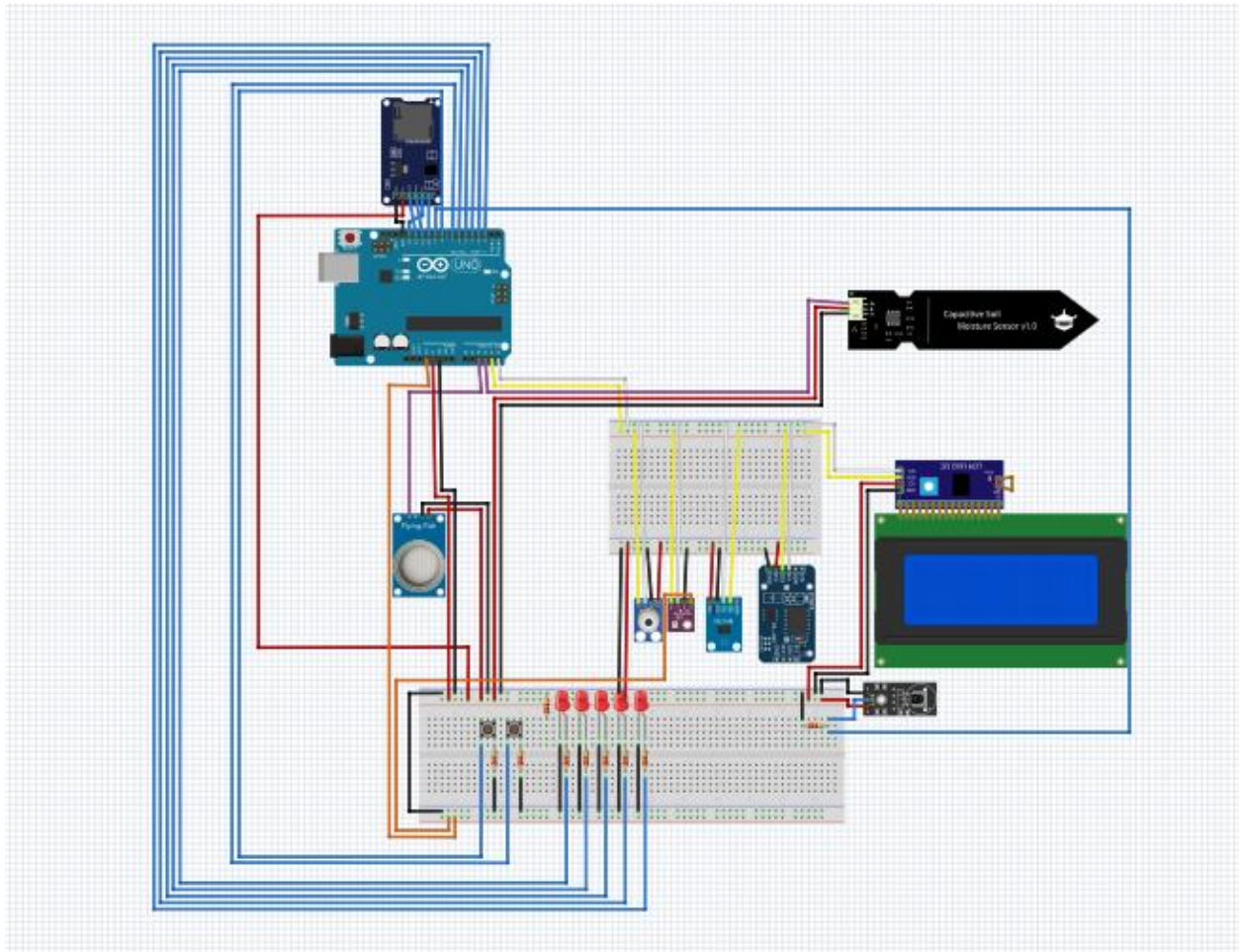


Analista de Sistemas Año 1

Prácticas Profesionalizantes 1

Proyecto: Estación Meteorológica



Integrantes: Facundo Neiel Molina, Franco Lugo, Ivan Ruiz, Mauro Diaz

Año: 2025

estacion_meteorologica.ino

```
// =====
// ESTACIÓN METEOROLÓGICA - PROGRAMA PRINCIPAL
// =====
// Control central de sensores, pantalla y almacenamiento.
// Lectura modular, navegación y registro optimizado.
// =====

// =====
// LIBRERÍAS PRINCIPALES
// =====
#include <Wire.h>      // Comunicación I2C
#include "lcd.h"       // Pantalla LCD
#include "navegacion.h" // Botones y control de pantallas

// =====
// MÓDULOS ACTIVOS
// =====
#include "bmp280.h"    // Temp, presión y altitud
#include "ds18b20.h"   // Temp del agua
#include "csmsv2.h"    // Humedad del suelo
#include "bh1750.h"    // Luz ambiental
#include "mq135.h"     // Gases / CO2
#include "gy906.h"     // Temp infrarroja
#include "ds3231.h"    // Reloj en tiempo real
#include "microsd.h"   // Registro en tarjeta SD
#include "leds_guia.h" // Indicadores LED por pantalla

// =====
// CONFIGURACIÓN INICIAL
// =====
void setup() {
  Serial.begin(9600);
  Wire.begin();

  iniciarSD();          // Inicia microSD
  iniciarPantalla();    // LCD
  iniciarLedsPantalla(); // LEDs
  cambiarLedPantalla(1); // LED inicial
  iniciarRTC();         // Reloj
}
```

```

    iniciarBMP();           // BMP280
    iniciarBH1750();        // BH1750
    iniciarGY906();         // Si está conectado
    iniciarDS18B20();       // DS18B20
    iniciarCSMSV2();        // Humedad suelo
    iniciarMQ135();         // MQ135
    iniciarNavegacion();    // Botones
    calibrarMQ135();        // Calibración gases
    mostrarPantalla(1);     // Pantalla inicial
}

// =====
// BUCLE PRINCIPAL
// =====
void loop() {
    leerNavegacion();       // Botones / cambio de pantallas
    leerRTC();              // Hora y fecha
    leerBMP();              // Presión y altitud
    leerBH1750();           // Luz
    leerGY906();            // Infrarrojo
    leerDS18B20();          // Temp agua
    leerCSMSV2();           // Humedad tierra
    leerMQ135();            // CO2
    escribirSD("registro.txt"); // Guarda datos
    cambiarLedPantalla(pantallaActual); // LED activo
    delay(1000);            // Actualización cada segundo
}

```

lcd.h

```

#ifndef _LCD_H
#define _LCD_H

// =====
// LIBRERÍAS
// =====
#include <Arduino.h>
#include <LiquidCrystal_I2C.h>

// =====

```

```

// VARIABLES EXTERNAS DE SENSORES
// =====
extern float temp_bmp280, presion_bmp280, altitud_bmp280;
extern float temp_ds18b20;
extern uint8_t humedad_tierra;
extern float ratio;
extern String calidad, tendencia;
extern float lux;
extern float TEMP_OBJETO_GY906;
extern String FECHA, HORARIO, DIA_SEMANA;

// =====
// LCD CONFIGURACIÓN
// =====
LiquidCrystal_I2C lcd(0x27, 20, 4); // Dir I2C y tamaño (20x4)

// =====
// FUNCIONES DE CONTROL LCD
// =====

// Pantalla de bienvenida
inline void pantallaInicio() {
    lcd.clear();
    lcd.setCursor(4, 0); lcd.print("!Bienvenidos");
    lcd.setCursor(8, 1); lcd.print("a la");
    lcd.setCursor(6, 2); lcd.print("Estacion");
    lcd.setCursor(4, 3); lcd.print("Meteorologica!");
    delay(3000);
}

// =====
// PANTALLAS DE INFORMACIÓN
// =====

inline void pantallaBMP280() {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("<====BMP280====>");
    lcd.setCursor(0, 1); lcd.print("Temp: "); lcd.print(temp_bmp280, 1); lcd.print(" C");
    lcd.setCursor(0, 2); lcd.print("Presion: "); lcd.print(presion_bmp280, 1); lcd.print(" hPa");
    lcd.setCursor(0, 3); lcd.print("Altitud: "); lcd.print(altitud_bmp280, 1); lcd.print(" m");
}

inline void pantallaDS18_CSMS() {

```

```

    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("<=====DS18B20=====>");
    lcd.setCursor(0, 1); lcd.print("Temp Agua: "); lcd.print(temp_ds18b20, 1); lcd.print(" C");
    lcd.setCursor(0, 2); lcd.print("<=====CSMSV2=====>");
    lcd.setCursor(0, 3); lcd.print("Humedad: "); lcd.print(humedad_tierra); lcd.print(" %");
}

inline void pantallaMQ135() {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("<=====MQ135=====>");
    lcd.setCursor(0, 1); lcd.print("CO2: "); lcd.print(ratio, 1); lcd.print("%");
    lcd.setCursor(0, 2); lcd.print("Estado: "); lcd.print(calidad);
    lcd.setCursor(0, 3); lcd.print("Tendencia: "); lcd.print(tendencia);
}

inline void pantallaBH_GY() {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("<=====BH1750=====>");
    lcd.setCursor(0, 1); lcd.print("Lux: "); lcd.print(lux, 1);
    lcd.setCursor(0, 2); lcd.print("<=====GY906=====>");
    lcd.setCursor(0, 3); lcd.print("Temp IR: "); lcd.print(TEMP_OBJETO_GY906, 1); lcd.print(" C");
}

inline void pantallaRTC() {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("<=====FECHA=====>");
    lcd.setCursor(0, 1); lcd.print("Dia: "); lcd.print(DIA_SEMANA); lcd.print(" ");
    lcd.print(FECHA);
    lcd.setCursor(0, 2); lcd.print("<=====HORARIO=====>");
    lcd.setCursor(0, 3); lcd.print("Hora: "); lcd.print(HORARIO);
}

// =====
// INICIALIZACIÓN
// =====
inline void iniciarPantalla() {
    lcd.begin(20, 4);
    lcd.backlight();
    lcd.clear();
    pantallaInicio();
}

// =====

```

```

// CAMBIO DE PANTALLAS
// =====
uint8_t pantallaActual = 0;

void mostrarPantalla(uint8_t id) {
    pantallaActual = id;
    switch (id) {
        case 0: pantallaBMP280(); break;
        case 1: pantallaDS18_CSMS(); break;
        case 2: pantallaMQ135(); break;
        case 3: pantallaBH_GY(); break;
        case 4: pantallaRTC(); break;
    }
}

#endif

```

Navegación.h

```

#ifndef _NAVEGACION_H_
#define _NAVEGACION_H_

#include <Arduino.h>
#include "lcd.h" // Para usar mostrarPantalla()

// =====
// CONFIGURACIÓN DE PINES DE LOS BOTONES
// =====
// Usa resistencias pull-up internas.
// Los botones se conectan entre el pin y GND.
#define BTN_IZQ 4
#define BTN_DER 3

// =====
// VARIABLES DE CONTROL (Antirrebote)
// =====
unsigned long ultimoMovimiento = 0;

```

```
const unsigned long debounceDelay = 250; // milisegundos entre movimientos

// =====
// VARIABLES DE CONTROL DE PANTALLA
// =====
extern uint8_t pantallaActual; // declarada en lcd.h
const uint8_t totalPantallas = 5; // cantidad total de pantallas (ajustá si cambian)

// =====
// INICIALIZACIÓN DE BOTONES
// =====
inline void iniciarNavegacion() {
    pinMode(BTN_IZQ, INPUT_PULLUP);
    pinMode(BTN_DER, INPUT_PULLUP);
    // Serial.println(F("Navegación por botones iniciada correctamente."));
}

// =====
// LECTURA DE BOTONES Y NAVEGACIÓN
// =====
inline void leerNavegacion() {
    unsigned long ahora = millis();
    if (ahora - ultimoMovimiento < debounceDelay)
        return;

    // Botón IZQUIERDA
    if (!digitalRead(BTN_IZQ)) {
        if (pantallaActual == 0) pantallaActual = totalPantallas - 1;
        else pantallaActual--;
        mostrarPantalla(pantallaActual);
        ultimoMovimiento = ahora;
        return;
    }

    // Botón DERECHA
    if (!digitalRead(BTN_DER)) {
        pantallaActual = (pantallaActual + 1) % totalPantallas;
        mostrarPantalla(pantallaActual);
        ultimoMovimiento = ahora;
        return;
    }
}
```

```
#endif
```

```
Leds_guia.h
```

```
#ifndef _LCD_GUIA_H
```

```
#define _LCD_GUIA_H
```

```
#include "lcd.h" // Tu archivo donde inicializás LiquidCrystal
```

```
// =====
```

```
// CONTROL DE LEDS SEGÚN LA "PANTALLA" ACTUAL
```

```
// =====
```

```
// Cada pantalla del LCD tendrá un LED indicador asociado.
```

```
// Por ejemplo, si mostrás distintas lecturas de sensores,
```

```
// podés prender el LED correspondiente a la pantalla activa.
```

```
#define LED_PANTALLA1 6
```

```
#define LED_PANTALLA2 5
```

```
#define LED_PANTALLA3 4
```

```
#define LED_PANTALLA4 3
```

```
#define LED_PANTALLA5 2
```

```
// Variables globales
```

```
static unsigned long ultimoParpadeo = 0;
```

```
static bool estadoLED = false;
```

```
extern uint8_t pantallaActual; // declarada en lcd.h
```

```
// -----
```

```
// Inicializa los pines de los LEDs como salidas
```

```
// -----
```

```
void iniciarLedsPantalla() {
```

```
    const uint8_t leds[5] = {LED_PANTALLA1, LED_PANTALLA2, LED_PANTALLA3,  
LED_PANTALLA4, LED_PANTALLA5};
```

```
    for (uint8_t i = 0; i < 5; i++) {
```

```
        pinMode(leds[i], OUTPUT);
```

```
        digitalWrite(leds[i], LOW);
```

```
    }
```

```
}
```

```
// -----
```

```
// Cambia la pantalla activa y enciende su LED correspondiente
```

```
// -----
```

```
void cambiarLedPantalla(uint8_t nuevaPantalla) {
    if (nuevaPantalla < 1 || nuevaPantalla > 5) return;
    pantallaActual = nuevaPantalla;

    // Apaga todos los LEDs
    digitalWrite(LED_PANTALLA1, LOW);
    digitalWrite(LED_PANTALLA2, LOW);
    digitalWrite(LED_PANTALLA3, LOW);
    digitalWrite(LED_PANTALLA4, LOW);
    digitalWrite(LED_PANTALLA5, LOW);

    // Enciende solo el LED de la pantalla activa
    switch (pantallaActual) {
        case 1: digitalWrite(LED_PANTALLA1, HIGH); break;
        case 2: digitalWrite(LED_PANTALLA2, HIGH); break;
        case 3: digitalWrite(LED_PANTALLA3, HIGH); break;
        case 4: digitalWrite(LED_PANTALLA4, HIGH); break;
        case 5: digitalWrite(LED_PANTALLA5, HIGH); break;
    }
}

#endif
```

MicroSD.h

```
#ifndef _MICROSD_H
#define _MICROSD_H

// =====
// MÓDULO MICROSD - LECTURA Y ESCRITURA DE DATOS
// =====
// Este módulo maneja la escritura de los datos de los sensores en una
// tarjeta microSD y la lectura opcional de registros. También utiliza
```

```

// la memoria EEPROM para conservar el número de registro más reciente.
// =====

// -----
// LIBRERÍAS NECESARIAS
// -----
#include <Arduino.h>
#include <SPI.h>    // Comunicación SPI (Serial Peripheral Interface)
#include <SD.h>     // Librería oficial de Arduino para manejo de tarjetas SD
#include <EEPROM.h> // Permite leer y escribir en la memoria EEPROM interna

// -----
// 🌡️ VARIABLES EXTERNAS DE SENSORES
// -----
// Se traen únicamente las variables necesarias desde los módulos de
// sensores, para reducir el peso total del programa. Cada variable es
// declarada como 'extern' porque su definición real está en otro archivo.
extern float temp_bmp280, presion_bmp280, altitud_bmp280; // BMP280 → temperatura,
presión y altitud
extern float temp_ds18b20;                               // DS18B20 → temperatura del agua
extern uint8_t humedad_tierra;                           // CSMSV2 → humedad del suelo
extern float ratio;                                       // MQ-135 → relación de CO2
extern String calidad, tendencia;                        // MQ-135 → estado y tendencia del aire
extern float lux;                                         // BH1750 → luz ambiental (lux)
extern float TEMP_OBJETO_GY906;                          // GY-906 → temperatura infrarroja
extern String FECHA, HORARIO;                            // DS3231 → fecha y hora actual

// -----
// CONFIGURACIÓN DEL MÓDULO MICROSD
// -----

// Pin Chip Select (SS) del módulo microSD.
// Nota: este valor puede variar según la placa o el módulo utilizado.
// Ejemplo común: 10 para Arduino UNO, 5 para ESP32, etc.
#define SD_CS_PIN 10

// Contador de registros (se conserva el valor entre reinicios usando EEPROM)
int registroID = 0;

// Objeto global para manejar archivos en la tarjeta SD
File archivo;

// -----

```

```

// FUNCIÓN: iniciarSD()
// -----
// Inicializa la tarjeta microSD y carga el último ID guardado desde EEPROM.
// Devuelve:
// true  → si la inicialización fue exitosa
// false → si falló la inicialización
// -----
bool iniciarSD() {
    // Serial.println(F("Iniciando tarjeta microSD...")); // DEBUG opcional

    // Intentar inicializar la tarjeta (descomentá si necesitás verificar)
    /*
    if (!SD.begin(SD_CS_PIN)) {
        Serial.println(F("Error: no se pudo inicializar la tarjeta SD"));
        return false;
    }
    */

    // Leer último ID guardado desde la EEPROM
    EEPROM.get(0, registroID);

    // Si el valor leído no es válido, comenzar desde 1
    if (registroID < 1)
        registroID = 1;

    // Serial.println(F("Tarjeta microSD lista para usar")); // DEBUG opcional
    return true;
}

// -----
// FUNCIÓN: escribirSD(const char *nombreArchivo)
// -----
// Escribe en la tarjeta SD los valores actuales de los sensores.
// Crea el archivo si no existe, o agrega información al final si ya existe.
//
// Parámetros:
// nombreArchivo → nombre del archivo donde guardar (ej: "datos.txt")
//
// Devuelve:
// true  → si la escritura fue exitosa
// false → si hubo error al abrir o escribir el archivo
// -----
bool escribirSD(const char *nombreArchivo) {

```

```
// Abrir archivo en modo escritura (crea si no existe)
archivo = SD.open(nombreArchivo, FILE_WRITE);

if (archivo) {
    // =====
    // BLOQUE DE ESCRITURA DE DATOS
    // =====

    // --- ID del registro ---
    archivo.print("Registro ID: ");
    archivo.println(String(registroID++)); // Incrementa para el siguiente registro

    // --- Fecha y hora del RTC ---
    archivo.print("Hora y Fecha: ");
    archivo.println(ler RTC()); // Función proveniente del módulo DS3231

    // --- Sensor BMP280 ---
    archivo.print("BMP280 → ");
    archivo.print("Temp: "); archivo.print(temp_bmp280); archivo.print(" °C ");
    archivo.print("Pres: "); archivo.print(presion_bmp280); archivo.print(" hPa ");
    archivo.print("Alti: "); archivo.print(altitud_bmp280); archivo.println(" m");

    // --- Sensor DS18B20 ---
    archivo.print("DS18B20 → ");
    archivo.print("Temp. Agua: "); archivo.print(temp_ds18b20); archivo.println(" °C");

    // --- Sensor CSMSV2 ---
    archivo.print("CSMSV2 → ");
    archivo.print("Humedad Tierra: "); archivo.print(humedad_tierra); archivo.println(" %");

    // --- Sensor BH1750 ---
    archivo.print("BH1750 → ");
    archivo.print("Luz: "); archivo.print(lux); archivo.println(" lx");

    // --- Sensor MQ-135 ---
    archivo.print("MQ-135 → ");
    archivo.print("CO2: "); archivo.print(ratio); archivo.print("% | ");
    archivo.print("Estado: "); archivo.print(calidad);
    archivo.print(" | Tendencia: "); archivo.println(tendencia);

    // --- Sensor GY-906 ---
    archivo.print("GY-906 → ");
```

```

    archivo.print("Temp. Infrarroja: "); archivo.print(TEMP_OBJETO_GY906); archivo.println("
    °C");

    // =====
    // FINALIZACIÓN
    // =====
    archivo.close(); // Siempre cerrar tras escribir
    EEPROM.put(0, registroID); // Guardar nuevo ID en EEPROM

    // Serial.print(F("Datos escritos en ")); Serial.println(nombreArchivo); // DEBUG
    return true;
}

// Si el archivo no se pudo abrir:
// Serial.print(F("Error al abrir ")); Serial.println(nombreArchivo); // DEBUG
return false;
}

// -----
// FUNCIÓN OPCIONAL: leerSD(const char *nombreArchivo)
// -----
// Lee el contenido completo de un archivo de la tarjeta SD y lo muestra
// por el monitor serie. Ideal para depuración o pruebas.
// -----
/*
void leerSD(const char *nombreArchivo) {
    archivo = SD.open(nombreArchivo);

    if (archivo) {
        Serial.print(F("Contenido de ")); Serial.println(nombreArchivo);
        Serial.println(F("-----"));

        while (archivo.available()) {
            Serial.write(archivo.read());
        }

        archivo.close();
        Serial.println(F("\n-----"));
        Serial.println(F("Lectura finalizada"));
    } else {
        Serial.print(F("Error al abrir ")); Serial.println(nombreArchivo);
    }
}

```

```
*/
```

```
#endif
```

```
Ds3231.h
```

```
#ifndef _DS3231_H
```

```
#define _DS3231_H
```

```
#include <Arduino.h>
```

```
#include <RTCLib.h>
```

```
// ===== OBJETO DEL RTC =====
```

```
RTC_DS3231 rtc;
```

```
DateTime fecha;
```

```
// ===== VARIABLES GLOBALES =====
```

```
int DIA, MES, ANIO, HORA, MINUTO, SEGUNDO;
```

```
String DIA_SEMANA, FECHA, HORARIO;
```

```
// ===== INICIO =====
```

```
inline void iniciarRTC() {
```

```
    // Wire.begin(); // Si no se inicializa en otro módulo
```

```
    // Inicializa y ajusta automáticamente con la hora de compilación
```

```
    rtc.begin();
```

```
    rtc.adjust(DateTime(__DATE__, __TIME__));
```

```
}
```

```
// ===== NOMBRE DEL DÍA =====
```

```
inline String obtenerNombreDia(uint8_t diaSemana) {
```

```
    switch (diaSemana) {
```

```
        case 1: return "Dom";
```

```
        case 2: return "Lun";
```

```
        case 3: return "Mar";
```

```
        case 4: return "Mie";
```

```
        case 5: return "Jue";
```

```
        case 6: return "Vie";
        case 7: return "Sab";
        default: return "???";
    }
}

// ===== LECTURA =====
inline String leerRTC() {
    fecha = rtc.now();

    DIA = fecha.day();
    MES = fecha.month();
    ANIO = fecha.year();
    HORA = fecha.hour();
    MINUTO = fecha.minute();
    SEGUNDO = fecha.second();
    DIA_SEMANA = obtenerNombreDia(fecha.dayOfTheWeek() + 1);

    char buffer[35];
    sprintf(buffer, "%s %02d/%02d/%04d %02d:%02d:%02d",
        DIA_SEMANA.c_str(), DIA, MES, ANIO, HORA, MINUTO, SEGUNDO);

    return String(buffer);
}

#endif
```

Bh1750.h

```
#ifndef _BH1750_H
#define _BH1750_H

#include <Arduino.h>
#include <BH1750.h>

// ===== SENSOR =====
BH1750 sensorBH1750; // Comunicación I2C (SDA, SCL)

// ===== VARIABLES =====
float lux = 0.0F; // Nivel de luz ambiental en lux

// ===== INICIO =====
inline void iniciarBH1750() {
    // Wire.begin(); // Si no se inicializa en otro módulo
    sensorBH1750.begin();
}

// ===== LECTURA =====
inline void leerBH1750() {
    lux = sensorBH1750.readLightLevel();
}

#endif
```

Bmp280.h

```
#ifndef _BMP280_H
#define _BMP280_H

#include <Arduino.h>
#include <Adafruit_BMP280.h> // Librería del sensor BMP280

// ===== SENSOR =====
Adafruit_BMP280 bmp; // Comunicación I2C (SDA, SCL)

// ===== VARIABLES =====
static float temp_bmp280 = 0.0F;
static float presion_bmp280 = 0.0F;
static float altitud_bmp280 = 0.0F;
static float p0_bmp = 0.0F; // Presión base (nivel del mar)

// ===== UMBRALES =====
#define TEMP_MAX 35.0F
#define PRESION_MIN 950.0F
#define PRESION_MAX 1050.0F
#define ALTITUD_MAX 300.0F

// ===== INICIO =====
inline void iniciarBMP() {
    // Dirección I2C 0x76 (usa 0x77 según módulo)
    // if (!bmp.begin(0x76)) while (1); // Para debug
    p0_bmp = bmp.readPressure() / 100.0F;
}

// ===== LECTURA =====
inline void leerBMP() {
    temp_bmp280 = bmp.readTemperature();
    presion_bmp280 = bmp.readPressure() / 100.0F; // Pa → hPa
    altitud_bmp280 = 44330 * (1.0 - pow(presion_bmp280 / 1013.25, 0.1903));
}

#endif
```

Csmsv2.h

```
#ifndef _CSMSV2_H
#define _CSMSV2_H

#include <Arduino.h>

// ===== CONFIGURACIÓN =====
#define PIN_SENSOR_TIERRA A0 // Pin analógico del sensor
#define HUMEDAD_MIN_SEGURA 25 // Mínimo aceptable (%)
#define HUMEDAD_MAX_SEGURA 90 // Máximo aceptable (%)
const int AIR_VALUE = 588; // Valor en aire seco
const int WATER_VALUE = 308; // Valor en agua

// ===== VARIABLE =====
static uint8_t humedad_tierra = 0; // % de humedad actual

// ===== INICIO =====
inline void iniciarCSMSV2() {
    pinMode(PIN_SENSOR_TIERRA, INPUT);
}

// ===== LECTURA =====
inline void leerCSMSV2() {
    int lectura = analogRead(PIN_SENSOR_TIERRA); // Leer valor ADC
    humedad_tierra = map(lectura, AIR_VALUE, WATER_VALUE, 0, 100);
    humedad_tierra = constrain(humedad_tierra, 0, 100);
}

#endif
```

Ds18b20.h

```
#ifndef _DS18B20_H
#define _DS18B20_H

#include <Arduino.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// ===== CONFIGURACIÓN =====
#define DS18B20_PIN 2 // Pin de datos OneWire
#define TEMP_MAX_DS18B20 37.0 // Temperatura máxima segura
#define TEMP_MIN_DS18B20 -10.0 // Temperatura mínima opcional

// ===== OBJETOS =====
static OneWire oneWire(DS18B20_PIN); // Comunicación OneWire
static DallasTemperature sensors(&oneWire); // Controlador del sensor

// ===== VARIABLE =====
static float temp_ds18b20 = 0.0; // Temperatura actual (°C)

// ===== INICIO =====
inline void iniciarDS18B20() {
    sensors.begin(); // Inicia comunicación
}

// ===== LECTURA =====
inline void leerDS18B20() {
    sensors.requestTemperatures(); // Solicita medición
    temp_ds18b20 = sensors.getTempCByIndex(0); // Lee temperatura
}

#endif
```

Gy906.h

```
#ifndef _GY906_H
#define _GY906_H

#include <Adafruit_MLX90614.h>

// =====
// OBJETO DEL SENSOR
// =====
static Adafruit_MLX90614 gy906;

// =====
// VARIABLES DE MEDICIÓN
// =====
// Notas:
// - Se usa 'static' para limitar el alcance al archivo (evita duplicaciones)
// - Se usa 'float' porque la librería devuelve ese tipo
static float TEMP_AMBIENTE_GY906 = 0;
static float TEMP_OBJETO_GY906 = 0;

// =====
// FUNCIONES
// =====

// Inicializa el sensor (llamar una sola vez)
inline void iniciarGY906() {
  if (!gy906.begin()) {
    Serial.println(F("Error GY-906"));
    while (true); // Error crítico
  }
  Serial.println(F("GY-906 OK"));
}

// Lee las temperaturas ambiente y de objeto
inline void leerGY906() {
  TEMP_AMBIENTE_GY906 = gy906.readAmbientTempC();
  TEMP_OBJETO_GY906 = gy906.readObjectTempC();
}

#endif
```

Mq135.h

```
#ifndef _MQ135_H
#define _MQ135_H

#include <Arduino.h>

// ===== CONFIGURACIÓN =====
constexpr uint8_t MQ135_PIN = A2; // Pin analógico del sensor
constexpr float RL = 1.0; // Resistencia de carga (kΩ)
constexpr float CLEAN_AIR_RATIO = 9.8; // RS/RO en aire limpio

// ===== VARIABLES =====
float RO = 10.0; // Resistencia base (aire limpio)
float RS = 0.0; // Resistencia actual del sensor
float ratio = 0.0; // Relación RS/RO
float ratio_anterior = 0;
String calidad = "Desconocida";
String tendencia = "Estable";

// ===== CALIBRACIÓN =====
inline void calibrarMQ135() {
    float RS_sum = 0.0;

    // Promedia 50 lecturas en aire limpio
    for (uint8_t i = 0; i < 50; i++) {
        int adc = analogRead(MQ135_PIN);
        if (adc == 0) adc = 1;
        float rs = RL * (1023.0 / adc - 1.0);
        RS_sum += rs;
        delay(100);
    }

    RS = RS_sum / 100.0;
    RO = RS / CLEAN_AIR_RATIO;
}

// ===== LECTURA =====
inline float leerMQ135() {
    int adc = analogRead(MQ135_PIN);
```

```
if (adc == 0) adc = 1;

RS = RL * (1023.0 / adc - 1.0);
ratio = RS / RO;

// Clasificación del aire
if (ratio > 3.0)    calidad = F("Excelente");
else if (ratio > 2.0) calidad = F("Buena");
else if (ratio > 1.5) calidad = F("Regular");
else if (ratio > 1.0) calidad = F("Mala");
else              calidad = F("Crítica");

// Tendencia
if (ratio > ratio_anterior + 0.1)    tendencia = F("Empeorando ↑");
else if (ratio < ratio_anterior - 0.1) tendencia = F("Mejorando ↓");
else                                tendencia = F("Estable →");

ratio_anterior = ratio;
return ratio;
}

// ===== INICIO =====
inline void iniciarMQ135() {
    pinMode(MQ135_PIN, INPUT);
    calibrarMQ135();
}

#endif
```